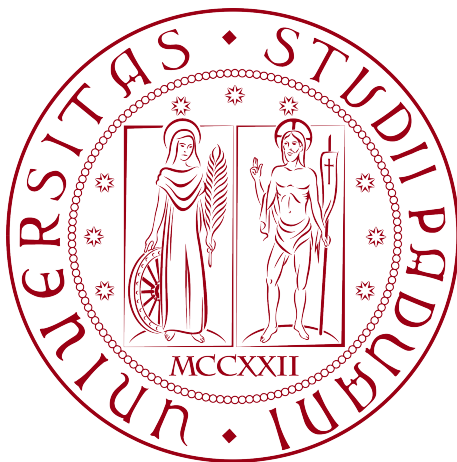


Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA "TULLIO LEVI-CIVITA"

CORSO DI LAUREA IN INFORMATICA



**Sviluppo di un Middleware basato su Spring per
l'Integrazione di Flussi di Dati**

Tesi di Laurea Triennale

Relatore

Prof. Zanella Marco

Laureando

Baldo Leonardo

Matricola 2042372

Ringraziamenti

Desidero esprimere la mia gratitudine al professor Zanella Marco, mio relatore, per l'aiuto e il sostegno che mi ha dato durante la stesura dell'elaborato.

Vorrei anche ringraziare, con affetto, i miei genitori per il loro sostegno, il grande aiuto e la loro presenza in ogni momento durante gli anni di studio.

Desidero poi ringraziare i miei amici per i bellissimi anni trascorsi insieme e le mille avventure vissute.

Padova, 27 agosto 2026

Baldo Leonardo

Abstract

Il presente documento descrive il lavoro svolto durante il periodo di stage, dalla durata di trecentoventi ore, dal laureando Baldo Leonardo presso l'azienda SyncLab S.r.l.

Il progetto di stage svolto riguarda lo studio e l'implementazione di *Middleware*, in particolar modo il *framework* Spring. L'applicazione da sviluppare dovrà leggere i dati da un Topic Kafka, arricchirli con dati presenti su un *database* e caricare il dato finale su un altro Topic Kafka.

Prima di iniziare con lo sviluppo effettivo del progetto è stato svolto uno studio approfondito delle tecnologie e degli strumenti coinvolti, i quali verranno analizzati nei successivi capitoli.

Indice

1	Introduzione	1
1.1	L'azienda	1
1.2	Organizzazione del testo	1
1.2.1	Struttura del testo	1
1.2.2	Convenzioni tipografiche	2
2	Descrizione dello Stage	3
2.1	Introduzione al progetto	3
2.2	Organizzazione del lavoro	3
2.3	Analisi preventiva dei rischi	4
3	Analisi dei Requisiti	6
3.1	Introduzione	6
3.2	Obiettivi dello stage	6
3.3	Tracciamento dei requisiti	7
3.4	Tabella dei requisiti	9
3.5	Ambito del confronto con Apache Flink	10
3.6	Ambito dei test di performance	11
4	Middleware	12
4.1	Introduzione al concetto di Middleware	12
4.2	Definizione e ruolo del Middleware	12
4.3	Classificazione dei Middleware	13
4.4	Enterprise Integration Patterns	14
4.4.1	Content Enricher	14
4.4.2	Dead Letter Channel	15
4.4.3	Altri pattern rilevanti	16
4.5	Apache Kafka come servizio di messaggistica	17
4.5.1	Introduzione e caratteristiche generali	17
4.5.2	Event streaming	17
4.5.3	Architettura e concetti fondamentali	18
4.5.4	Persistenza dei messaggi	20

4.5.5 Semantica di consegna	21
4.6 Spring come piattaforma <i>middleware</i>	21
4.6.1 Spring Boot	21
4.6.1.1 Auto-configuration	22
4.6.1.2 Convention over configuration	22
4.6.2 Spring Data JPA	23
4.6.3 Spring Kafka	23
4.7 Apache Flink come paradigma alternativo	24
4.7.1 DataStream API	24
4.7.2 Gestione nativa dello stato	24
4.7.3 Async I/O	25
4.7.4 Confronto concettuale con l'approccio Spring adottato	25
4.8 Valutazione delle prestazioni	25
4.8.1 Metriche di valutazione	26
4.8.2 Metodologia di test	26
5 Codifica	28
5.1 Tecnologie e strumenti	28
5.1.1 Linguaggio e piattaforma	28
5.1.1.1 Java	28
5.1.1.2 Apache Maven	29
5.1.2 Framework applicativo	30
5.1.2.1 Spring	31
5.1.3 Infrastruttura di supporto	32
5.1.3.1 Docker	32
5.1.3.2 PostgreSQL	33
5.1.3.3 Apache Kafka	34
5.2 Progettazione	36
5.2.1 Modello dei dati	36
5.2.2 Progettazione dei topic Kafka	36
5.2.3 Progettazione del flusso di elaborazione	37
5.3 Implementazione	38
5.3.1 Main	38
5.3.2 Modello dei dati	39
5.3.3 Accesso ai dati	41
5.3.4 Logica di enrichment	42
5.3.5 Consumer	44
5.3.6 Producer	45
5.3.7 Sintesi delle annotazioni utilizzate	47
5.4 Progettazione di una pipeline in Apache Flink	48

5.4.1 Sorgente e formato dei dati	48
5.4.2 Fase di enrichment	48
5.4.3 Gestione degli errori	48
5.4.4 Destinazione e garanzie di consegna	49
5.5 Strumenti per i test di performance	49
5.5.1 Generazione del carico	49
5.5.2 Osservazione del comportamento del sistema	49
5.5.3 Osservabilità applicativa	49
5.5.4 Ambiente di test	50
6 Conclusioni	51
6.1 Sintesi del lavoro svolto	51
6.2 Conoscenze acquisite	51
6.3 Limiti e sviluppi futuri	52
Acronimi	I
Glossario	II
Bibliografia	VI

Elenco delle figure

1.1 Synclab S.r.l.	1
4.1 Content Enricher	14
4.2 Dead Letter Channel	15
4.3 Message Channel	16
4.4 Point-to-Point Channel	16
4.5 Message Translator	17
4.6 Partition	20
5.1 Java	28
5.2 Apache Maven	30
5.3 Spring	31
5.4 Docker	32
5.5 PostgreSQL	34
5.6 Apache Kafka	35
5.7 Flusso di elaborazione	37

Elenco delle tabelle

3.1 Tabella del tracciamento dei requisiti funzionali	9
3.2 Tabella del tracciamento dei requisiti qualitativi	10
3.3 Tabella del tracciamento dei requisiti di vincolo	10
4.1 Confronto tra l'approccio Spring e il paradigma Flink . .	25
5.1 Tabella dei topic	36
5.2 Tabella delle annotazioni	47

Elenco dei codici

5.1 EnrichmentMiddlewareApplication	38
5.2 Customer	39
5.3 OrderEvent	40
5.4 EnrichedOrder	41
5.5 CustomerRepository	41
5.6 EnrichmentService	42
5.7 OrderConsumer	44
5.8 EnrichedProducer	46

Capitolo 1

Introduzione

1.1 L'azienda

L'esperienza di tirocinio svolta dal laureando ha avuto luogo negli uffici della sede veronese dell'azienda SyncLab S.r.l.¹

Sync Lab è una Innovative Company attenta ai paradigmi della trasformazione digitale che realizza prodotti e soluzioni per diversi mercati quali: Sanità, Industria, Energia, Telco, Finanza e Trasporti & Logistica.

L'offerta di consulenza specialistica, sintesi di circa 20 anni nel settore IT e la forte competenza in diversi domini tecnologici, aiutano il cliente a dominare i nuovi trend e le sfide che ne derivano, quali: GDPR, Big Data, Cloud Computing, IoT, Mobile e Cyber Security. In questo scenario Sync Lab si posiziona come uno dei più importanti system integrator nel panorama italiano delle aziende ICT.



Figura 1.1: SyncLab S.r.l.

1.2 Organizzazione del testo

1.2.1 Struttura del testo

Il presente documento è suddiviso in sei capitoli il cui contenuto è brevemente riassunto in seguito:

¹SyncLab S.r.l. - URL: <https://synclab.it/home>

Il secondo capitolo è composto da un'introduzione al progetto, dall'organizzazione del testo e dall'analisi preventiva dei rischi

Il terzo capitolo è dedicato agli obiettivi e all'analisi dei requisiti

Il quarto capitolo spiega la parte teorica del progetto

Il quinto capitolo spiega la parte pratica del progetto

Il sesto capitolo contiene le conclusioni

1.2.2 Convenzioni tipografiche

In merito alla redazione del presente documento, sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini ambigui o di uso non comune menzionati vengono definiti nel glossario, situato alla fine del presente documento;
- i termini particolarmente rilevanti in una sezione e quelli in lingua straniera non di uso comune o facenti parti del gergo tecnico sono evidenziati con il carattere *corsivo*, fatta eccezione per le occorrenze presenti nei titoli delle sezioni o nelle didascalie;
- i comandi di terminale, i frammenti di codice sorgente e i nomi di file o *directory* sono evidenziati con il carattere monospaziato.

Capitolo 2

Descrizione dello Stage

2.1 Introduzione al progetto

Lo scopo principale di questo tirocinio sarà lo studio e l'implementazione di Middleware. Principalmente verrà sviluppata con il framework Spring (in particolare SpringBoot, Spring Data/JPA e Sprint-Batch), inoltre confrontarla con la piattaforma Apache Flink, se possibile.

L'applicazione da sviluppare dovrà leggere i dati da un Topic Kafka, arricchirli con dati presenti su PostgreSQL e caricare il dato finale su un altro Topic Kafka. Pertanto, il compito dello studente sarà quello di studiare, progettare, sviluppare e implementare tutte queste funzionalità sulle tecnologie sopra riportate.

2.2 Organizzazione del lavoro

Il lavoro è stato suddiviso in due fasi principali: una prima fase dedicata allo studio delle tecnologie e degli strumenti da utilizzare, in particolare del framework Spring, e una seconda fase dedicata all'implementazione vera e propria.

Il tirocinio si è svolto prevalentemente in modalità remota, con presenza in azienda una volta a settimana per confrontarmi con il tutor aziendale sull'andamento del progetto e per individuare eventuali criticità da correggere.

2.3 Analisi preventiva dei rischi

Per ogni rischio vengono indicati **descrizione** e **soluzione**, inoltre la **probabilità** che esso accada e l'**impatto** che il danno causerebbe.

1. Curva di apprendimento ripida

Probabilità: Alta (quasi certo che accada con stack così ampio).

Impatto: Medio (mitigabile con una fase di studio iniziale).

Descrizione: il progetto prevede l'utilizzo di tecnologie e strumenti di cui non si ha esperienza (Spring Boot, Spring Data/JPA, Spring Batch, Apache Kafka, PostgreSQL, eventualmente Apache Flink); questo può comportare rallentamenti dovuti allo studio e all'adozione delle nuove tecnologie, riducendo di conseguenza il tempo disponibile per lo sviluppo.

Soluzione: il periodo di tirocinio prevede una prima parte di studio delle tecnologie e degli strumenti che si andranno ad utilizzare, con particolare attenzione al framework Spring.

2. Ambiente di sviluppo

Probabilità: Media (problemi di setup sono comuni).

Impatto: Medio (risolvibili rapidamente con Docker).

Descrizione: è necessario un ambiente che includa Kafka, PostgreSQL e tutti gli strumenti utilizzati, configurato in modo da funzionare in modo affidabile e da rendere eventuali modifiche facilmente ripristinabili.

Soluzione: implementando il progetto tramite Docker, risulta più semplice e rapido il setup e l'integrazione di ogni strumento.

3. Integrazione tra tecnologie eterogenee

Probabilità: Alta (cuore tecnico del progetto).

Impatto: Alto (compromissione funzionalità principali).

Descrizione: il flusso dati tra Apache Kafka e PostgreSQL comporta problemi tipici dei sistemi distribuiti: gestione della concorrenza, consistenza dei dati, latenza nelle query di arricchimento e gestione dei fallimenti parziali.

Soluzione: definire una strategia di error handling.

4. Gestione dati inconsistenti o mancanti durante l'arricchimento

Probabilità: Media (prevedibile in sistemi di arricchimento dati).

Impatto: Medio (con scarto l'impatto resta contenuto).

Descrizione: può verificarsi il caso in cui il dato proveniente da Apache Kafka faccia riferimento a un record non presente in PostgreSQL; è quindi necessario definire una politica di gestione di questi casi.

Soluzione: in caso di dati errati o non trovati, questi vengono scartati e viene generata una notifica tramite messaggio di errore.

5. Confronto con Apache Flink

Probabilità: Alta (molto probabile per vincoli di tempo).

Impatto: Basso (opzionale, quindi di impatto contenuto).

Descrizione: Flink adotta un modello di programmazione molto diverso da Spring; il confronto tra le due tecnologie richiede una comprensione approfondita di entrambi i paradigmi per risultare significativo e non banale.

Soluzione: tale attività viene indicata come obiettivo desiderabile e non obbligatorio.

Capitolo 3

Analisi dei Requisiti

3.1 Introduzione

L'analisi dei requisiti è un'attività preliminare allo sviluppo di un sistema software, il cui scopo è quello di definire le funzionalità che il nuovo prodotto deve offrire, ovvero i requisiti che devono essere soddisfatti dal software sviluppato. L'analisi dei requisiti è di fondamentale importanza, in quanto essa guida le successive fasi di sviluppo del prodotto.

3.2 Obiettivi dello stage

Notazione

Si farà riferimento ai requisiti secondo le seguenti notazioni:

- **O** per i requisiti obbligatori, vincolanti in quanto obiettivo primario richiesto dal committente;
- **D** per i requisiti desiderabili, non coinvolgenti o strettamente necessari, ma dal riconoscibile valore aggiunto;
- **F** per i requisiti facoltativi, rappresentanti valore aggiunto non strettamente competitivo.

Le sigle precedentemente indicate saranno seguite da una coppia sequenziale di numeri, identificativo del requisito.

Obiettivi fissati

Si prevede lo svolgimento dei seguenti obiettivi:

- Obbligatori
 - **O01**: Studio framework Spring;
 - **O02**: Implementazione flusso mediante Spring;
 - **O03**: Relazione tecnica che descriva le scelte tecniche prese durante il percorso e le motivazioni a riguardo.
- Desiderabili
 - **D01**: Studio Apache Flink;
 - **D02**: Implementazione flusso mediante Apache Flink;
 - **D03**: Affrontare un'analisi comparativa usando metriche ben precise e coerenti con il contesto applicativo.
- Facoltativi
 - **F01**: Effettuare dei test di performance mediante strumenti di misurazione dedicati;
 - **F02**: Implementare il flusso anche su tecnologie alternative (Apache Camel, Apache NiFi).

3.3 Tracciamento dei requisiti

Da un'analisi degli obiettivi e dei requisiti effettuata sul progetto è stata stilata la tabella che traccia i requisiti. Sono stati individuati diversi tipi di requisiti e si è quindi fatto utilizzo di un codice identificativo per distinguerli. Il codice dei requisiti, dove ogni requisito è identificato con il carattere **R**, è strutturato nel seguente modo:

R[tipologia][importanza]-[numero]

In particolare le tipologie sono:

- **F** per i requisiti funzionali;
- **Q** per i requisiti qualitativi;
- **V** per i requisiti di vincolo.

In particolare le importanze sono:

- **O** per i requisiti obbligatori;
- **D** per i requisiti desiderabili;
- **F** per i requisiti facoltativi.

3.4 Tabella dei requisiti

Requisito	Descrizione	Stato
RFO-01	Il sistema deve leggere (consumare) messaggi da un Topic Kafka sorgente	Soddisfatto
RFO-02	Il sistema deve interrogare un <i>database</i> per recuperare dati di arricchimento correlati al messaggio ricevuto	Soddisfatto
RFO-03	Il sistema deve combinare (arricchire) il dato originale ricevuto da Apache Kafka con i dati letti dal <i>database</i>	Soddisfatto
RFO-04	Il sistema deve pubblicare (produrre) il dato arricchito su un Topic Kafka di destinazione, diverso da quello sorgente	Soddisfatto
RFO-05	Il sistema deve gestire la connessione e configurazione verso Apache Kafka (consumer e producer) tramite Spring	Soddisfatto
RFD-06	Il sistema deve gestire gli errori di elaborazione, come dato non trovato sul <i>database</i> , prevedendo una strategia	Soddisfatto
RFD-07	Deve essere realizzata un'implementazione equivalente delle stesse funzionalità utilizzando Apache Flink, per permettere il confronto	Parzialmente soddisfatto
RFF-08	Il sistema deve essere valutabile tramite un insieme definito di metriche di performance (<i>throughput</i> , <i>latenza</i> , <i>consumer lag</i>)	Insoddisfatto

Tabella 3.1: Tabella del tracciamento dei requisiti funzionali

Requisito	Descrizione	Stato
RQD-01	Il codice deve essere organizzato in modo modulare, con separazione tra strato di ingestion, arricchimento e produzione, per favorire manutenzione e test	Soddisfatto
RQD-02	Le due implementazioni (Spring vs Flink) devono essere progettate in modo da poter essere confrontate su metriche omogenee	Insoddisfatto

Tabella 3.2: Tabella del tracciamento dei requisiti qualitativi

Requisito	Descrizione	Stato
RVO-01	Deve essere utilizzato il framework Spring	Soddisfatto
RVO-02	La sorgente dati di input è vincolata a essere un Topic Kafka	Soddisfatto
RVO-03	L'output deve essere vincolato a un Topic Kafka di destinazione (diverso da quello di input)	Soddisfatto

Tabella 3.3: Tabella del tracciamento dei requisiti di vincolo

3.5 Ambito del confronto con Apache Flink

Tra gli obiettivi desiderabili (3.2) figurano **D01** (studio di Apache Flink), **D02** (implementazione equivalente) e **D03** (analisi comparativa), tracciati dai requisiti **RFD-07** e **RQD-02**. La priorità nel tempo a disposizione del tirocinio è assegnata agli obiettivi obbligatori **O01-O03**: l'implementazione equivalente in Flink (**D02**) non viene per questo eseguita.

Lo studio di Flink (**D01**) e l'analisi comparativa (**D03**) vengono invece condotti a livello teorico: i concetti fondamentali della piattaforma e il confronto concettuale con l'approccio Spring adottato sono trattati nella sezione 4.7, mentre la progettazione di come la medesima *pipeline* sarebbe strutturata con Flink, sui contratti dei messaggi e sui *topic* effettivamente implementati, è definita nella sezione 5.4. L'esecuzione di tale implementazione resta uno sviluppo futuro, riportato nelle conclusioni.

3.6 Ambito dei test di performance

Tra gli obiettivi facoltativi definiti in questo capitolo figura **F01** (Effettuare dei test di performance mediante strumenti di misurazione dedicati), tracciato dal requisito **RFF-08**. Trattandosi di un requisito facoltativo, la sua esecuzione non rientra nell'ambito vincolante del presente lavoro, come per Apahce Flink (??Ambito del confronto con Apache Flink)) la priorità è assegnata agli obiettivi obbligatori **001-003**, relativi allo studio e all'implementazione del *middleware* tramite Spring.

Il requisito non viene per questo trattato come assente: la metodologia di valutazione e gli strumenti concreti con cui si eseguirebbe la misurazione, sono definiti rispettivamente nelle sezioni 4.8 e 5.5. Ciò che resta fuori dall'ambito del presente lavoro è unicamente l'esecuzione materiale di tali test, riportata come sviluppo futuro nelle conclusioni.

Capitolo 4

Middleware

4.1 Introduzione al concetto di Middleware

Nell'ambito dei sistemi informativi moderni, la crescente necessità di integrare applicazioni eterogenee, spesso sviluppate con tecnologie diverse e distribuite su più nodi, ha reso centrale il ruolo del *middleware*. Con questo termine si indica genericamente quello strato *software* che si colloca tra il sistema operativo e le applicazioni, con lo scopo di semplificare la comunicazione, la gestione dei dati e l'interoperabilità tra componenti distinte di un sistema distribuito.

Il presente tirocinio si colloca proprio in questo ambito: l'obiettivo è studiare, progettare e implementare un *middleware* in grado di leggere dati da un topic Kafka, arricchirli con informazioni provenienti da un *database* relazionale PostgreSQL, e pubblicare il risultato su un secondo topic Kafka. Questo capitolo introduce i concetti teorici alla base del *middleware* in generale, per poi restringere il campo alle tecnologie specifiche impiegate nel progetto: Spring Boot, Spring Data JPA e Apache Kafka.

4.2 Definizione e ruolo del Middleware

Il *middleware* è un tipo di programma software che fornisce alle applicazioni software servizi che vanno oltre quelli offerti dal sistema operativo. È lo strato intermedio tra le applicazioni e le componenti sottostanti, come sistemi operativi o *database*, facilitando così la comunicazione, l'integrazione e la gestione delle risorse nei sistemi distribuiti. Il suo ruolo primario è quello di nascondere la

complessità dell'infrastruttura sottostante, garantendo interoperabilità, scalabilità e sicurezza.

Le caratteristiche principali sono:

- **Disaccoppiamento:** i sistemi produttori e consumatori di informazioni non devono obbligatoriamente conoscersi o condividere lo stesso linguaggio, piattaforma o formato dati.
- **Interoperabilità:** consente la comunicazione tra applicazioni scritte in linguaggi diversi o eseguite su piattaforme differenti.
- **Scalabilità e affidabilità:** molti *middleware* offrono tolleranza ai guasti e garanzie di consegna dei messaggi.

Il *middleware* sviluppato in questo progetto ha la funzione di ponte tra due flussi di eventi Kafka, arricchendo il dato in transito con informazioni recuperate da PostgreSQL. Si tratta quindi di un *middleware* orientato ai dati e agli eventi, piuttosto che di un semplice bus di comunicazione RPC.

4.3 Classificazione dei Middleware

Sono state definite numerose categorie di *middleware*, in base al settore in cui vengono impiegate o al modulo applicativo che servono. Una bibliografia recente ha identificato le principali categorie di *middleware* come segue:

- **Transactional Middleware:** elaborazione di transazioni multiple sincrone/asincrone, fungendo da cluster di richieste associate provenienti da sistemi distribuiti (es. transazioni bancarie o pagamenti con carta di credito).
- **Message-Oriented Middleware (MOM):** basati sullo scambio di messaggi asincroni attraverso code o topic (es. Apache Kafka).
- **Procedural Middleware:** architetture remote e locali per connettersi, trasmettere e recuperare le risposte software delle comunicazioni tra sistemi asincroni (operazione di chiamata).
- **Object-Oriented Middleware:** simile al *middleware* procedurale, ma incorpora i principi di progettazione della programmazione

orientata agli oggetti. Dal punto di vista analitico, i suoi componenti software comprendono riferimenti agli oggetti, eccezioni ed ereditarietà delle proprietà tramite richieste di oggetti distribuiti.

Il progetto oggetto del tirocinio si colloca nella seconda categoria: si tratta infatti di un *Message-Oriented Middleware* con funzionalità di arricchimento dati (*data enrichment*), in cui il canale di comunicazione è rappresentato da Kafka e la logica di trasformazione è delegata a un componente applicativo (Spring).

4.4 Enterprise Integration Patterns

Per inquadrare correttamente il problema dal punto di vista architeturale, è utile fare riferimento agli *Enterprise Integration Patterns* (EIP), un catalogo di *pattern* ricorrenti nell'integrazione di sistemi enterprise, che permette di descrivere il problema del progetto in termini standard e riconosciuti in letteratura.

4.4.1 Content Enricher

Il *pattern* più rilevante per questo lavoro è il *Content Enricher* (mostrato in Fig. 4.1): un componente che riceve un messaggio contenente informazioni parziali e lo arricchisce con dati aggiuntivi provenienti da una fonte esterna (*database* relazionale), producendo un messaggio più completo da inoltrare al passo successivo della *pipeline*.

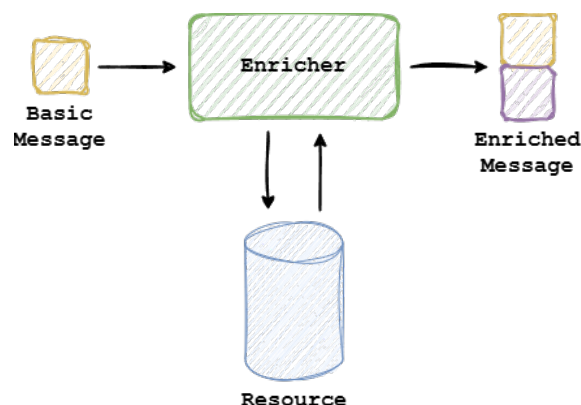


Figura 4.1: Content Enricher

Nell'implementazione sviluppata, questo *pattern* è realizzato concretamente dalla classe `EnrichmentService`, che riceve in input un

`OrderEvent` (contenente, tra gli altri campi, un `customerId`), interroga PostgreSQL tramite `CustomerRepository` (Spring Data JPA) per recuperare i dati del cliente corrispondente, e produce in output un `EnrichedOrder`, ovvero il messaggio arricchito con le informazioni mancanti.

4.4.2 Dead Letter Channel

Un secondo *pattern*, altrettanto rilevante, è il Dead Letter Channel (mostrato in Fig. 4.2). Questo *pattern* affronta un problema tipico dei sistemi di integrazione basati su messaggi: cosa fare quando un messaggio non può essere elaborato correttamente? Le opzioni alternative sono entrambe insoddisfacenti:

- Bloccare la *pipeline*, in attesa che il problema venga risolto manualmente, con il rischio di fermare l'elaborazione di tutti i messaggi successivi (in Kafka, il blocco su una partizione impedisce l'avanzamento dell'*offset* per l'intera partizione);
- Scartare silenziosamente il messaggio, con conseguente perdita di dati e nessuna possibilità di analisi o recupero successivo.

Il *pattern Dead Letter Channel* propone invece una terza via: i messaggi che non possono essere elaborati vengono instradati su un canale separato (la *dead letter queue*, o DLQ), dedicato esclusivamente alla raccolta dei messaggi "falliti". Questo consente al flusso principale di proseguire senza interruzioni, mantenendo al contempo traccia dei messaggi problematici per un'analisi o un riprocessamento successivo.

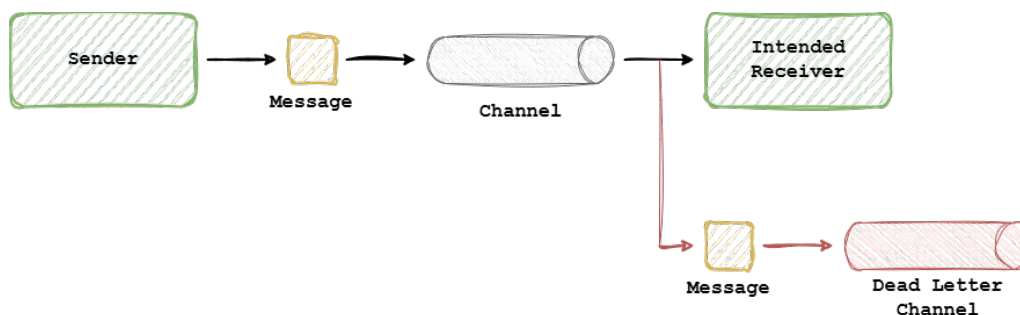


Figura 4.2: Dead Letter Channel

Nel progetto, questo *pattern* è implementato nel modo seguente: quando `EnrichmentService` tenta di arricchire un `OrderEvent` con un `customerId` non presente su PostgreSQL, viene sollevata una `CustomerNotFound`

`dException`. Questa eccezione viene intercettata e gestita instradando il messaggio originale sul topic `orders-input-dlq`, anziché proseguire verso `EnrichedProducer` e il topic `orders-output`. In questo modo, un singolo dato incompleto o malformato non compromette l'elaborazione degli ordini successivi, e resta comunque disponibile per un'ispezione a posteriori (ad esempio tramite la Kafka UI predisposta nell'ambiente di sviluppo).

4.4.3 Altri pattern rilevanti

Altri *pattern* EIP utili e applicabili al progetto sono:

- **Message Channel:** collega le applicazioni tramite un canale di messaggi, in cui un'applicazione scrive le informazioni nel canale e l'altra le legge dal canale. I messaggi qui rappresentati dai topic Kafka (`orders-input`, `orders-output`, `orders-input-dlq`).

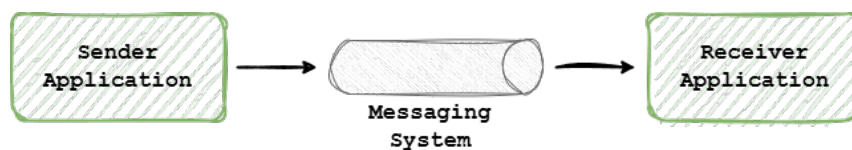


Figura 4.3: Message Channel

- **Point-to-Point Channel:** invia il messaggio su un canale *Point-to-Point*, che garantisce che solo un destinatario riceva un determinato messaggio.

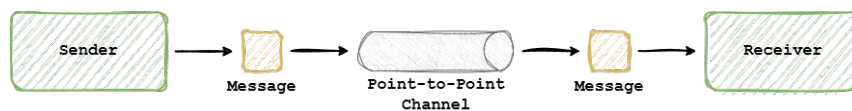


Figura 4.4: Point-to-Point Channel

- **Message Translator:** utilizza un filtro speciale (*Translator*), tra altri filtri o applicazioni per convertire un formato di dati in un altro. Nel progetto è usato nella conversione tra `OrderEvent` e `EnrichedOrder`, entrambi DTO distinti nel package `dto/`.

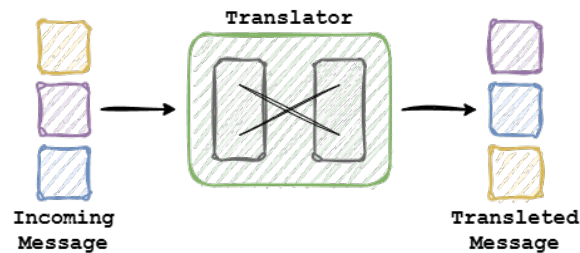


Figura 4.5: Message Translator

4.5 Apache Kafka come servizio di messaggistica

4.5.1 Introduzione e caratteristiche generali

Apache Kafka è una piattaforma distribuita per lo streaming di eventi, originariamente sviluppata da LinkedIn¹ e successivamente rilasciata come progetto open source sviluppata da Apache Software Foundation².

A differenza dei tradizionali sistemi di messaggistica basati su code (come i classici Message Queue), Kafka nasce con l'obiettivo di gestire flussi di dati ad alto *Throughput*, bassa latenza e con garanzie di persistenza e ripetibilità della lettura, caratteristiche che lo rendono particolarmente adatto sia a scenari di semplice trasporto di messaggi, sia a scenari più complessi di *event streaming* ed *event sourcing*.

Nel contesto di questo progetto, Kafka non è impiegato semplicemente come canale di trasporto, ma come componente architetturale a tutti gli effetti: da esso dipendono sia l'ingresso dei dati grezzi nel sistema, sia l'uscita dei dati arricchiti.

4.5.2 Event streaming

L'*event streaming* è l'equivalente digitale del sistema nervoso centrale del corpo umano. Rappresenta il fondamento tecnologico di un mondo "sempre attivo", in cui le aziende sono sempre più definite dal *software* e automatizzate, e in cui l'utente del *software* è a sua volta un *software*.

Dal punto di vista tecnico, lo streaming di eventi consiste in:

¹LinkedIn - URL: <https://www.linkedin.com>

²Apache Software Foundation - URL: <https://www.apache.org/>

- acquisizione in tempo reale di dati provenienti da fonti quali *database*, sensori, dispositivi mobili, servizi cloud e applicazioni software sotto forma di flussi di eventi;
- archiviazione duratura di tali flussi per un successivo recupero; nella manipolazione, elaborazione e risposta ai flussi di eventi sia in tempo reale che a posteriori;
- instradamento dei flussi di eventi verso diverse tecnologie di destinazione a seconda delle necessità.

L'*event streaming* garantisce quindi un flusso continuo e l'interpretazione dei dati, in modo che le informazioni giuste si trovino nel posto giusto, al momento giusto.

4.5.3 Architettura e concetti fondamentali

Producer e Consumer I produttori sono quelle applicazioni client che pubblicano (scrivono) eventi su Kafka, mentre i consumatori sono quelle che si abbonano (leggono ed elaborano) a tali eventi. In Kafka, produttori e consumatori sono completamente disaccoppiati e indipendenti l'uno dall'altro, il che costituisce un elemento chiave del design per ottenere l'elevata scalabilità (i produttori non devono mai attendere i consumatori).

Nel *middleware* sviluppato, l'applicazione stessa riveste entrambi i ruoli: agisce come consumer rispetto al topic sorgente (da cui legge i dati grezzi da arricchire) e come producer rispetto al topic di destinazione (su cui scrive il dato arricchito).

Consumer Group Kafka introduce il concetto di consumer group, un insieme di consumer identificati da uno stesso *group.id*, che si ripartiscono automaticamente le partizioni di un topic, così che ciascun messaggio venga elaborato da un solo consumer all'interno del gruppo. Questo meccanismo consente di scalare orizzontalmente il *middleware*, aggiungendo istanze aggiuntive dell'applicazione senza necessità di logica di bilanciamento del carico esplicita: sarà Kafka stesso a ridistribuire le partizioni tra i consumer disponibili (fenomeno noto come *rebalance*).

Event Gli eventi vengono organizzati e archiviati in modo permanente in argomenti (topic). Un argomento è simile a una cartella in un file system, mentre gli eventi sono i file contenuti in quella cartella.

Gli argomenti in Kafka sono sempre multi-producer e multi-consumer : un argomento può avere zero, uno o più produttori che vi scrivono eventi, così come zero, uno o più consumatori che si abbonano a tali eventi. Gli eventi in un argomento possono essere letti tutte le volte che è necessario: a differenza dei sistemi di messaggistica tradizionali, gli eventi non vengono eliminati dopo la lettura. È l'utente a definire per quanto tempo Kafka debba conservare gli eventi tramite un'impostazione di configurazione specifica per ogni argomento, trascorso il quale gli eventi più vecchi verranno eliminati. Le prestazioni di Kafka rimangono sostanzialmente costanti indipendentemente dalla dimensione dei dati, quindi l'archiviazione dei dati per lunghi periodi di tempo non presenta alcun problema.

Topic L'unità logica fondamentale di Kafka è il topic, un canale a cui i producer inviano messaggi e da cui i consumer li leggono.

Ogni topic è fisicamente suddiviso in una o più partition, che rappresentano l'unità di parallelismo: ogni partizione è una sequenza ordinata e immutabile di messaggi, identificati da un numero progressivo detto offset. Questa distribuzione dei dati è molto importante per la scalabilità, poiché consente alle applicazioni client di leggere e scrivere i dati da e verso molti broker contemporaneamente. Quando un nuovo evento viene pubblicato su un argomento, viene effettivamente aggiunto a una delle partizioni dell'argomento (mostrato in Fig. 4.6).

Per garantire la tolleranza ai guasti e l'alta disponibilità dei dati, ogni argomento può essere replicato, anche tra diverse aree geografiche o data center, in modo che ci siano sempre più broker in possesso di una copia dei dati nel caso in cui si verificano problemi, si desideri eseguire operazioni di manutenzione sui broker e così via. Un'impostazione comune in produzione è un fattore di replica pari a 3, ovvero ci saranno sempre tre copie dei dati. Questa replica viene eseguita a livello di partizioni degli argomenti.

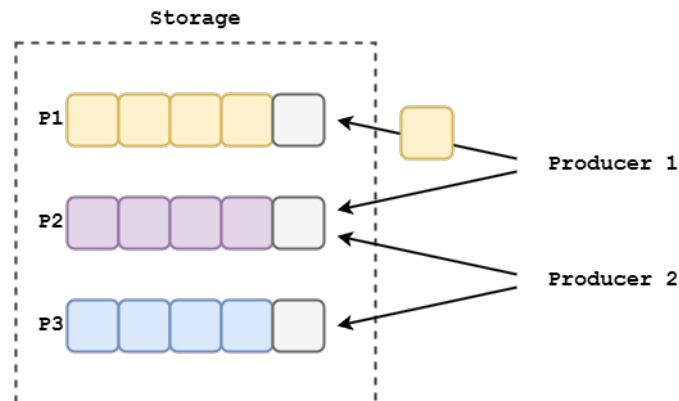


Figura 4.6: Partition

Broker e Cluster Un cluster Kafka è composto da uno o più broker, ciascuno responsabile della gestione di un sottoinsieme di partizioni. La distribuzione dei dati su più broker, unita alla replica delle partizioni (controllata dal parametro `replication factor`), garantisce tolleranza ai guasti: in caso di indisponibilità di un broker, un'altra replica può assumere il ruolo di leader per la partizione interessata, senza perdita di dati.

4.5.4 Persistenza dei messaggi

Una delle caratteristiche distintive di Kafka rispetto ai tradizionali MOM è la persistenza dei messaggi su disco, per un periodo di retention configurabile, indipendentemente dal fatto che siano già stati letti da un consumer.

Questo consente di:

- rileggere lo storico dei messaggi in caso di errore o necessità di riprocessamento;
- gestire il disaccoppiamento temporale tra producer e consumer, che non devono essere necessariamente attivi nello stesso momento;
- garantire resilienza del *middleware* in caso di malfunzionamento temporaneo della componente di arricchimento (indisponibilità del *database*), poiché i messaggi restano disponibili nel topic finché non vengono correttamente processati.

4.5.5 Semantica di consegna

Dal punto di vista della semantica di consegna, Kafka supporta tre modalità:

- **At-most-once**: il messaggio può essere perso, ma non elaborato più volte (offset committato prima dell'elaborazione).
- **At-least-once**: il messaggio viene sempre elaborato, ma può essere elaborato più volte in caso di errore o riavvio (offset committato dopo l'elaborazione). È la modalità più comune nei sistemi che, come quello del progetto, non tollerano perdita di dati.
- **Exactly-once**: garantisce che ogni messaggio venga elaborato esattamente una volta, tramite meccanismi di transazioni Kafka.

Nel contesto del *middleware* sviluppato, la scelta della semantica di consegna è un aspetto progettuale rilevante: poiché la fase di arricchimento comporta un'interrogazione al *database* PostgreSQL, occorre valutare attentamente le implicazioni di un'eventuale elaborazione duplicata del medesimo messaggio.

4.6 Spring come piattaforma *middleware*

Spring è tra i framework più diffusi in ambito enterprise per la costruzione di applicazioni Java robuste, modulari e facilmente integrabili con infrastrutture di messaggistica e basi di dati.

4.6.1 Spring Boot

Spring Boot rappresenta l'evoluzione del framework Spring pensata per semplificare la configurazione e l'avvio di applicazioni Java, tramite il paradigma dell'auto-configuration e della convention over configuration.

Alla base del framework resta il principio di *Inversion of Control* (IoC), realizzato tramite *Dependency Injection*: i componenti applicativi non istanziano direttamente le proprie dipendenze, ma le ricevono da un contenitore (l'*application context*), che ne gestisce il ciclo di vita.

Ai fini della realizzazione di un *middleware*, questo approccio offre alcuni vantaggi rilevanti:

- disaccoppiamento tra i vari componenti logici del sistema (lettura dei messaggi, logica di arricchimento, scrittura dei messaggi), che possono essere sviluppati, testati e sostituiti in modo indipendente;
- integrazione facile e veloce con le principali infrastrutture *enterprise* (*broker* di messaggistica, basi di dati, sistemi di monitoraggio), grazie a un ricco ecosistema di starter e librerie ufficiali;
- riduzione del codice infrastrutturale a carico dello sviluppatore, che può concentrarsi sulla logica di dominio.

4.6.1.1 Auto-configuration

La configurazione automatica di Spring Boot è una funzionalità che configura automaticamente i componenti dell'applicazione in base alle dipendenze disponibili.

Crea e configura automaticamente i servizi e le impostazioni necessari per l'applicazione. Riduce la configurazione manuale e accelera lo sviluppo.

La configurazione automatica funziona tramite l'annotazione `@EnableAutoConfiguration`.

4.6.1.2 Convention over configuration

Il principio *Convention over configuration* (noto anche come *coding by convention*) è un paradigma di progettazione software utilizzato dai framework che mira a ridurre il numero di decisioni che uno sviluppatore che utilizza il framework è tenuto a prendere, senza necessariamente rinunciare alla flessibilità e ai principi del *Don't Repeat Yourself* (DRY).

La frase significa essenzialmente che uno sviluppatore deve specificare solo gli aspetti non convenzionali dell'applicazione. Quando la convenzione implementata dallo strumento corrisponde al comportamento desiderato, il sistema si comporta come previsto senza dover scrivere file di configurazione. Solo quando il comportamento desiderato si discosta dalla convenzione implementata è necessaria una configurazione esplicita.

4.6.2 Spring Data JPA

Spring Data JPA è il modulo dell'ecosistema Spring dedicato alla persistenza dei dati, costruito sopra la specifica JPA (Java Persistence API) e sull'implementazione di riferimento Hibernate.

Il suo contributo principale, dal punto di vista architetturale, è l'astrazione dell'accesso ai dati tramite il *pattern* Repository. Lo sviluppatore definisce un'interfaccia che descrive le operazioni di accesso desiderate, senza doverne scrivere l'implementazione e le query SQL, che vengono generate automaticamente dal framework a partire dalle convenzioni di naming o da annotazioni dedicate.

Nel contesto di un *middleware* di enrichment, questo modulo assume un ruolo centrale: è il componente che permette di interrogare la fonte dati esterna (PostgreSQL) necessaria a completare le informazioni presenti nel messaggio in transito, disaccoppiando la logica di arricchimento dal dettaglio implementativo dell'accesso al *database*.

4.6.3 Spring Kafka

Per la parte di integrazione con Apache Kafka, l'ecosistema Spring mette a disposizione Spring Kafka, un modulo che fornisce un'astrazione ad alto livello sopra il client Kafka nativo, semplificando la configurazione dei producer e dei consumer e integrandoli pienamente con il modello di *Dependency Injection* tipico di Spring Boot.

Il concetto centrale su cui si fonda Spring Kafka è quello di *listener*: un componente applicativo si registra come consumatore di uno o più topic e viene invocato automaticamente ogni volta che un nuovo messaggio è disponibile, senza che lo sviluppatore lo debba gestire. Il modello di elaborazione risultante è quindi *event-driven* e continuo: l'applicazione resta costantemente in ascolto, reagendo agli eventi non appena questi si presentano, senza necessità di una schedulazione periodica.

Questo paradigma si adatta molto bene a scenari come quello affrontato nel presente lavoro, in cui il flusso di dati non ha una fine definita a priori (uno stream potenzialmente illimitato di eventi) e in cui è desiderabile minimizzare la latenza tra la produzione di

un evento e la sua elaborazione. Spring Kafka realizza , a livello applicativo, le astrazioni Kafka già introdotte nella sezione 4.5.3 (topic, consumer group, offset).

Questi elementi rendono Spring Kafka una scelta architetturale coerente con il *pattern* Content Enricher (descritto in 4.4.1): il *listener* funge da punto di ingresso del messaggio grezzo, delega la logica di arricchimento a un componente dedicato (tipicamente interfacciato con la base dati tramite Spring Data JPA), e infine si occupa della pubblicazione del messaggio arricchito sul topic di destinazione, chiudendo così il ciclo di elaborazione descritto nello schema architetturale generale del *middleware*.

4.7 Apache Flink come paradigma alternativo

Apache Flink è un motore Open Source di *stream processing* distribuito, progettato nativamente per l'elaborazione continua di flussi di eventi, a differenza dell'approccio a *listener* discreti adottato da Spring Kafka.

4.7.1 DataStream API

Il modello di programmazione principale di Flink è la **DataStream API**, un'interfaccia dichiarativa che definisce una *pipeline* come una sequenza di operatori applicati a un flusso continuo di eventi, anziché come un metodo invocato singolarmente per ciascun messaggio ricevuto (come avviene con `@KafkaListener`). Flink traduce questa sequenza in un grafo di esecuzione distribuito su più nodi (*task manager*), che elabora gli eventi in parallelo lungo l'intero *stream*.

4.7.2 Gestione nativa dello stato

A differenza dell'approccio adottato dal *middleware* sviluppato, in cui ogni evento è arricchito tramite un'interrogazione sincrona e senza stato (4.4.1), Flink offre un meccanismo di **stato nativo distribuito**, reso tollerante ai guasti tramite *Checkpointing* periodico. Questo permette, ad esempio, di mantenere in memoria una cache dei dati cliente più consultati, riducendo il numero di interrogazioni verso la fonte dati esterna.

4.7.3 Async I/O

Flink mette a disposizione un meccanismo di **Async I/O** per l'interazione con sistemi esterni senza bloccare il flusso di elaborazione in attesa di una risposta. A differenza di una query JPA sincrona, l'Async I/O invia più richieste concorrenti verso la base dati e riprende l'elaborazione di ciascun evento non appena la relativa risposta è disponibile, con un beneficio potenziale sul *Throughput* in scenari con latenza di rete o di *database* non trascurabile.

4.7.4 Confronto concettuale con l'approccio Spring adottato

La tabella seguente riassume le differenze tra l'approccio implementato con Spring e il paradigma Flink appena descritto.

Criterio	Spring (implementato)	Apache Flink (teorico)
Modello di elaborazione	<i>Listener event-driven</i> per singolo messaggio	<i>Stream processing</i> continuo su un grafo di operatori
Accesso al database	<i>Query</i> JPA sincrona per evento	Async I/O con <i>caching</i> tramite stato nativo
Gestione degli errori	<i>Try/catch</i> applicativo verso una DLQ	<i>Side output</i> nativo dell'operatore
Semantica di consegna	<i>At-least-once</i>	<i>Exactly-once</i> end-to-end
Complessità operativa	Contenuta, framework maturo e diffuso	Superiore, richiede un <i>cluster</i> Flink dedicato

Tabella 4.1: Confronto tra l'approccio Spring e il paradigma Flink

4.8 Valutazione delle prestazioni

La valutazione delle prestazioni di un *middleware* basato sul *pattern* Content Enricher segue una metodologia specifica, che tiene conto della caratteristica strutturale di questo *pattern*. Ogni evento elaborato comporta un'interrogazione verso una fonte dati esterna, che costituisce il candidato più probabile a collo di bottiglia dell'intera *pipeline*, a differenza di un semplice inoltro di messaggi tra due *topic*.

4.8.1 Metriche di valutazione

Si individuano quattro metriche, ciascuna informativa di un aspetto diverso del comportamento del sistema:

- **Throughput:** il numero di messaggi arricchiti con successo per unità di tempo, ovvero la capacità complessiva del *middleware* di sostenere un determinato flusso di eventi in ingresso;
- **Latenza *end-to-end*:** il tempo intercorso tra la pubblicazione di un evento grezzo sul *topic* sorgente (*orders-input*) e la comparsa del corrispondente evento arricchito sul *topic* di destinazione (*orders-output*). Si rileva non solo il valore medio, ma anche i percentili, poiché in sistemi di messaggistica la coda della distribuzione è spesso più significativa della media;
- **Consumer lag:** la differenza tra l'*offset* più recente scritto su un *topic* e l'*offset* più recente letto dal *consumer*. È una metrica particolarmente indicativa in questo contesto, poiché segnala se il *middleware* riesce a "stare al passo" con il flusso di eventi in ingresso o se accumula un ritardo progressivo;
- **Tempo di risposta della fonte dati esterna:** isolato dal resto della *pipeline*, misura specificamente il costo dell'interrogazione compiuta nella fase di *enrichment* (4.4.1).

4.8.2 Metodologia di test

La valutazione si articola in quattro scenari, di complessità crescente:

1. **Test di baseline:** un carico contenuto e costante, che stabilisce i valori di riferimento del sistema in condizioni ideali, lontane da qualunque saturazione.
2. **Test di carico crescente (*load test*):** un aumento progressivo e controllato del numero di messaggi pubblicati per secondo, fino a individuare il punto in cui il *consumer lag* smette di rimanere stabile e inizia a crescere in modo continuo, il limite della capacità di elaborazione in tempo reale del sistema.
3. **Test di stress:** un carico volutamente superiore alla capacità nominale individuata nello scenario precedente, per osservare se il sistema degrada in modo controllato (ad esempio accumulando

lag ma continuando a elaborare correttamente ogni messaggio) oppure manifesta comportamenti patologici (errori, *timeout* delle connessioni al *database*, blocco del *consumer*);

4. **Test con percentuale variabile di messaggi non arricchibili:** la quantità di `customerId` inesistenti nel flusso di ingresso viene fatta variare, per valutare l'impatto della gestione della Dead Letter Channel (descritta in 4.4.2) sulle prestazioni complessive, in particolare se l'instradamento verso `orders-input-dlq` introduca un *overhead* significativo rispetto al percorso verso `orders-output`.

Ciascuno scenario si ripete più volte, per distinguere un degrado sistematico delle prestazioni da normali variazioni dovute a fattori ambientali.

Capitolo 5

Codifica

5.1 Tecnologie e strumenti

5.1.1 Linguaggio e piattaforma

Il *middleware* è stato sviluppato in Java e gestito tramite Apache Maven come *build tool*, che si occupa della risoluzione delle dipendenze, della compilazione e del *packaging* dell'applicazione in un *jar* eseguibile.

5.1.1.1 Java

Java è un linguaggio di programmazione ad alto livello, orientato agli oggetti e a tipizzazione statica, che si appoggia sull'omonima piattaforma *software* di esecuzione, specificamente progettato per essere il più possibile indipendente dalla piattaforma *hardware* di esecuzione (tramite compilazione in *bytecode* prima e interpretazione poi da parte di una JVM).



Figura 5.1: Java

Valore aggiunto al progetto La tipizzazione statica di Java consente di individuare in fase di compilazione gran parte degli errori legati a un utilizzo scorretto dei contratti dei messaggi (`OrderEvent`, `EnrichedOrder`), un aspetto rilevante in un *middleware* in cui la correttezza del formato dei dati scambiati è centrale. Inoltre, l'ecosistema Java è quello su cui si basa nativamente l'intero *stack* tecnologico impiegato (Spring, i *client* ufficiali di Apache Kafka e i *driver* JDBC per PostgreSQL), garantendo la massima compatibilità e il supporto per ciascuna di queste componenti.

Punti deboli Rispetto a linguaggi più recenti pensati specificamente per microservizi *cloud-native*, Java presenta un tempo di avvio della JVM e un consumo di memoria a *runtime* superiori.

Competitor principali

- **C#:** principale linguaggio concorrente per lo sviluppo di applicazioni enterprise di questo tipo;
- **Kotlin:** in ambito JVM, quindi più facilmente compatibile con le librerie Java impiegate nel progetto.

Motivazione della scelta La scelta di Java è coerente con l'obiettivo del tirocinio, che prevedeva esplicitamente lo studio e l'implementazione del *middleware* tramite il framework Spring, basato su Java. Inoltre, la maggior integrazione di Java con Kafka e JPA lo rende una scelta più solida, in questo contesto specifico.

5.1.1.2 Apache Maven

Apache Maven è uno strumento di gestione di progetti *software* basati su Java e *build automation*.

Maven usa un costrutto conosciuto come POM; un file XML che descrive le dipendenze fra il progetto, le varie versioni di librerie necessarie e le dipendenze fra di esse. In questo modo si separano le librerie dalla *directory* di progetto utilizzando questo file descrittivo per definirne le relazioni.

Maven effettua automaticamente il *download* di librerie Java e *plugin* Maven dai vari *repository* definiti scaricandoli in locale o in un *repository* centralizzato lato sviluppo. Questo permette di recuperare in modo uniforme i vari file JAR e di poter spostare il progetto

indipendentemente da un ambiente all'altro avendo la sicurezza di utilizzare sempre le stesse versioni delle librerie.



Figura 5.2: Apache Maven

Valore aggiunto al progetto Maven ha permesso di dichiarare in modo esplicito e riproducibile l'intero insieme di dipendenze del progetto (`spring-kafka`, `spring-boot-starter-data-jpa`, driver `postgresql`, `Lombok`, ...), requisito importante per la riproducibilità dell'ambiente descritta anche a proposito di Docker.

Punti deboli Maven non dispone nativamente di un sistema di *caching* incrementale delle build avanzato quanto quello di alcuni concorrenti, aspetto che può diventare un limite in progetti di dimensioni maggiori con tempi di *build* più lunghi.

Competitor principali

- **Gradle:** utilizza uno *script* basato su Groovy o Kotlin al posto dell'XML e offre un sistema di *build* incrementale generalmente più *performante*.

Motivazione della scelta Maven è stato preferito a Gradle principalmente per la sua maggiore diffusione e documentazione nell'ambito specifico dei progetti Spring Boot, nonché per la maggiore semplicità del formato dichiarativo XML rispetto a uno *script* Groovy/Kotlin.

5.1.2 Framework applicativo

Il progetto si basa principalmente sul framework Spring, in particolare i seguenti moduli:

- **Spring Boot:** per semplificare la configurazione e l'avvio di applicazioni Java
- **Spring Data JPA:** per la persistenza dei dati
- **Spring Kafka:** per l'integrazione dichiarativa con Apache Kafka (consumer, producer, listener)

5.1.2.1 Spring

Spring è un framework Open Source per lo sviluppo di applicazioni su piattaforma Java. A questo framework sono associati tanti altri progetti, che hanno nomi composti come Spring Boot, Spring Data JPA e Spring Kafka. Questi progetti sono stati ideati per fornire funzionalità aggiuntive al framework.



Figura 5.3: Spring

Valore aggiunto al progetto Spring, tramite i suoi moduli specializzati, fornisce per ciascuna delle fasi tipiche del *middleware* un'astrazione dedicata: **Spring Boot**, **Spring Data JPA** , **Spring Kafka**.

Punti deboli Spring Boot ha un *footprint* di memoria e un tempo di avvio superiori rispetto a framework concorrenti pensati specificamente per ambienti *cloud-native* con *scaling* rapido, aspetto non critico per il *middleware* sviluppato, ma potenzialmente rilevante in un contesto di produzione con requisiti di scalabilità orizzontale molto stringenti.

Competitor principali

- **Quarkus** e **Micronaut**: principali *framework* concorrenti in ambito Java enterprise, entrambi progettati specificamente per ridurre tempo di avvio e consumo di memoria in ambienti containerizzati;
- **Jakarta EE**: standard enterprise su cui lo stesso Spring si è storicamente ispirato;
- **Apache Camel**: *framework* di integrazione che implementa nativamente numerosi *Enterprise Integration Patterns* (si veda 5.2).

Motivazione della scelta Spring è stato indicato esplicitamente come tecnologia di riferimento per l'attività di tirocinio, in coerenza con il suo ruolo di *framework enterprise* Java più diffuso e documentato.

5.1.3 Infrastruttura di supporto

L'ambiente di sviluppo è orchestrato tramite Docker e Docker Compose, secondo la definizione riportata nel file `docker-compose.yml`, che avvia tre servizi containerizzati:

- **PostgreSQL:** il *database* relazionale che ospita i dati di arricchimento;
- **Apache Kafka:** configurato in modalità KRaft (Kafka Raft), l'architettura più recente che elimina la dipendenza da Apache ZooKeeper per il coordinamento del *cluster*, unificando i ruoli di *broker* e *controller* in un singolo nodo (scelta adeguata a un ambiente di sviluppo locale a singolo nodo);
- **Kafka UI:** un'interfaccia web (esposta sulla porta `8081`) che consente di ispezionare topic, partizioni, messaggi e *consumer group* senza dover ricorrere alla sola riga di comando.

Questa infrastruttura consente di riprodurre in locale, in modo rapido e ripetibile, un ambiente rappresentativo di un *deployment* di produzione, mantenendo però la semplicità necessaria in fase di sviluppo e test.

5.1.3.1 Docker

Docker è un popolare *software* libero che utilizza la virtualizzazione a livello di sistema operativo per eseguire applicazioni in ambienti isolati chiamati *container*.

I container Docker sono un'estensione dei *container* del sistema operativo. Docker supporta sia i container Linux che quelli Windows. È possibile utilizzare Docker con container Linux anche nei sistemi Windows e MacOS grazie a tecniche di virtualizzazione.



Figura 5.4: Docker

Valore aggiunto al progetto Docker Compose ha permesso di definire, in un unico *file* dichiarativo, l'intero ambiente necessario al funzionamento del *middleware* (PostgreSQL, Apache Kafka e Kafka UI),

rendendolo riproducibile con un singolo comando su qualsiasi macchina di sviluppo, senza necessità di installare manualmente ciascun componente. Questo ha semplificato notevolmente sia lo sviluppo sia l'eventuale verifica del funzionamento del sistema, descritta in 5.3.

Punti deboli La virtualizzazione a livello di sistema operativo, per quanto più leggera rispetto a una macchina virtuale tradizionale, introduce comunque un certo *overhead* prestazionale rispetto all'esecuzione nativa. Inoltre, Docker richiede l'esecuzione di un *daemon* centrale, tipicamente con privilegi elevati, aspetto che pone alcune criticità di sicurezza note e mitigate solo parzialmente rispetto ad alternative più recenti.

Competitor principali

- **Podman:** compatibile con le stesse immagini e con gli stessi file *compose* di Docker, ma con un'architettura *daemonless* e supporto nativo per l'esecuzione senza privilegi di amministratore (*rootless*).

Motivazione della scelta Docker è stato scelto per la sua diffusione, garantendo la disponibilità di immagini ufficiali già pronte per tutti i componenti necessari ([postgres](#), [confluentinc/cp-kafka](#), [provectuslabs/kafka-ui](#)) e un'ampia documentazione. Rispetto a Podman, che pure avrebbe garantito funzionalità equivalenti con un modello di sicurezza migliore, Docker è stato preferito per la maggiore familiarità con lo strumento al momento dell'inizio del tirocinio.

5.1.3.2 PostgreSQL

PostgreSQL è un completo DBMS ad oggetti pubblicato con licenza libera (stile Licenza BSD). Spesso abbreviato come "Postgres", sebbene questo sia un nome vecchio dello stesso progetto, è una reale alternativa sia rispetto ad altri prodotti liberi come MySQL, Firebird SQL e MaxDB che a quelli a codice chiuso come Oracle, IBM Informix o DB2 ed offre caratteristiche uniche nel suo genere che lo pongono per alcuni aspetti all'avanguardia nel settore delle basi di dati.



Figura 5.5: PostgreSQL

Valore aggiunto al progetto La piena conformità ACID di PostgreSQL e la sua integrazione con Spring tramite *Spring Data JPA*/Hibernate hanno permesso di realizzare l'accesso ai dati senza scrivere *query* SQL esplicite (descritto in 5.3).

Punti deboli Per il caso d'uso specifico del progetto, la piena potenza relazionale di PostgreSQL risulta in parte sovradimensionata. Un *database* più leggero sarebbe stato sufficiente in un contesto di solo sviluppo o test.

Competitor principali

- **MySQL** e **MariaDB**: tra i DBMS *open source* relazionali più diffusi;
- **MongoDB**: *database* NoSQL documentale, per scenari in cui il modello dei dati fosse meno strutturato.

Motivazione della scelta PostgreSQL è stato scelto per la sua licenza libera, per la piena conformità agli standard SQL e ACID e per il supporto nativo offerto da Spring Data JPA/Hibernate. A differenza di un *database* NoSQL, la natura intrinsecamente strutturata e a schema fisso dei dati cliente (nome, email, città) rende inoltre un modello relazionale la scelta più naturale.

5.1.3.3 Apache Kafka

Apache Kafka è una piattaforma Open Source di *stream processing* scritta in Java e Scala e sviluppata dall'Apache Software Foundation. Il progetto mira a creare una piattaforma a bassa latenza ed alta velocità per la gestione di *feed* dati in tempo reale.

Questo progetto viene usato principalmente per tutte le applicazioni di elaborazioni di *stream* di dati in tempo reale.



Figura 5.6: Apache Kafka

Valore aggiunto al progetto Kafka costituisce l'infrastruttura di messaggistica su cui si fonda l'intero *middleware*, fungendo sia da sorgente (`orders-input`) sia da destinazione (`orders-output`, `orders-input-dlq`) del flusso di dati (descritto in 5.2). La sua caratteristica di persistenza dei messaggi su disco, unita al disaccoppiamento temporale tra produttori e consumatori, ha permesso di realizzare un sistema resiliente. Un'eventuale indisponibilità temporanea di PostgreSQL non comporta la perdita di alcun evento, poiché i messaggi restano disponibili sul *topic* finché non vengono correttamente elaborati.

Punti deboli Kafka introduce una complessità di configurazione e gestione superiore rispetto a un semplice *message broker*, complessità che, per un volume di messaggi contenuto come quello gestito nell'ambito del progetto, non viene pienamente ripagata dai suoi vantaggi distintivi (*Throughput* elevato, scalabilità orizzontale su più nodi).

Competitor principali

- **RabbitMQ** e **ActiveMQ**: *message broker* tradizionali orientati alle code piuttosto che allo *streaming* persistente di eventi;
- **Apache Pulsar**: piattaforma di *event streaming* più recente con un'architettura che separa *storage* e *serving*.

Motivazione della scelta La scelta di Apache Kafka è stata indicata esplicitamente come requisito del progetto di tirocinio. Rispetto ad alternative come RabbitMQ, Kafka offre nativamente la possibilità di rileggere lo storico dei messaggi (*replay*) grazie alla persistenza configurabile, caratteristica non scontata nei tradizionali *message broker* orientati alle code, dove un messaggio consumato viene tipicamente rimosso dalla coda.

5.2 Progettazione

Prima di procedere all'implementazione, sono state definite alcune scelte progettuali riguardanti il modello dei dati, la struttura dei Topic Kafka e la gestione degli errori.

5.2.1 Modello dei dati

Sono stati definiti tre tipi di oggetti, ciascuno con una responsabilità precisa:

- **OrderEvent**: rappresenta il messaggio grezzo in ingresso, contenente un identificativo dell'ordine (**orderId**), un riferimento al cliente (**customerId**) e i dati dell'ordine stesso (**product**, **amount**);
- **Customer**: entità contenente i dati del cliente (**fullName**, **email**, **city**) necessari all'arricchimento;
- **EnrichedOrder**: rappresenta il messaggio finale pubblicato in uscita, che unisce i dati dell'ordine originario con i dati del cliente recuperati da PostgreSQL, più un *timestamp* (**enrichedAt**) che documenta il momento dell'arricchimento.

Questa separazione segue il pattern Content Enricher: il messaggio in ingresso è "povero", mentre il contratto in uscita è "arricchito" con informazioni che il produttore originario dell'evento non possiede.

5.2.2 Progettazione dei topic Kafka

Sono stati previsti tre topic, ciascuno con un numero di partizioni proporzionato al ruolo previsto:

Topic	Partizioni	Ruolo
orders-input	3	ingresso degli ordini grezzi
orders-output	3	uscita degli ordini arricchiti
orders-input-dlq	1	dead letter queue per ordini non arricchibili

Tabella 5.1: Tabella dei topic

Il numero di partizioni maggiore per i topic di flusso principale (**orders-input**, **orders-output**) consente il parallelismo in lettura/scrittura

descritto in 4.5.2. La singola partizione per la DLQ riflette invece un volume atteso più basso e la priorità data alla semplicità di ispezione rispetto al parallelismo.

5.2.3 Progettazione del flusso di elaborazione

Il flusso di elaborazione è stato progettato secondo la seguente sequenza logica:

1. un evento ordine viene pubblicato su `orders-input`;
2. il middleware consuma il messaggio;
3. il middleware tenta di recuperare i dati del cliente corrispondente su PostgreSQL;
 - (a) se il cliente viene trovato, viene composto il messaggio arricchito e pubblicato su `orders-output`;
 - (b) se il cliente non viene trovato, l'evento originario viene instradato su `orders-input-dlq`, secondo il pattern Dead Letter Channel descritto in 4.4.2;

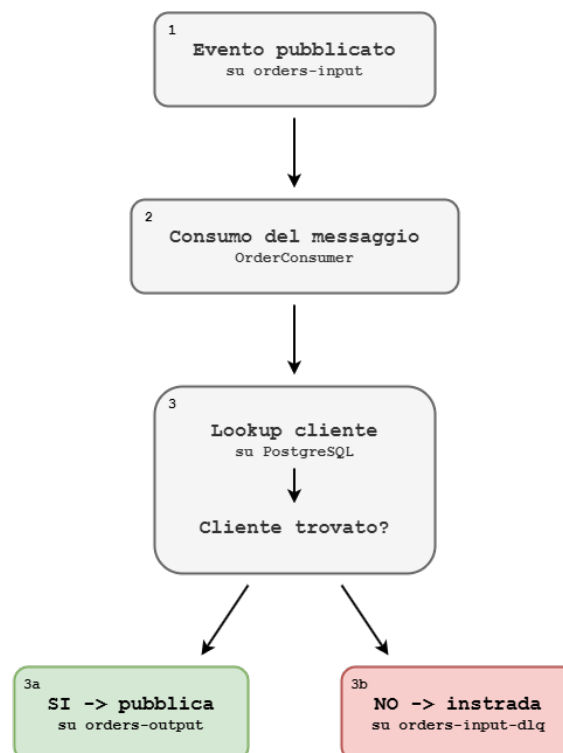


Figura 5.7: Flusso di elaborazione

Questa progettazione garantisce che, in caso di arresto anomalo dell'applicazione tra la lettura del messaggio e la sua elaborazione

completa, il messaggio venga riletto al riavvio (semantica at-least-once, discussa in 4.5.5), evitando così perdite di dati a scapito di un'eventuale elaborazione duplicata.

5.3 Implementazione

L'implementazione riprende le scelte progettuali descritte in 5.2, organizzandole in *package* distinti secondo una logica di separazione delle responsabilità. In questa sezione, oltre a presentare il codice, viene data particolare attenzione al significato e alla funzione delle Annotazioni impiegate, elemento caratterizzante dello stile dichiarativo tipico di Spring Boot.

5.3.1 Main

```
@SpringBootApplication
public class EnrichmentMiddlewareApplication {
    public static void main(String[] args) {
        SpringApplication
            .run(EnrichmentMiddlewareApplication.class, args);
    }
}
```

Codice 5.1: EnrichmentMiddlewareApplication

`@SpringBootApplication` è un'annotazione che ne racchiude tre:

- `@SpringBootConfiguration` (specializzazione di `@Configuration`): indica che la classe può definire Bean gestiti dal container Spring;
- `@EnableAutoConfiguration`: attiva il meccanismo di *auto-configuration* di Spring Boot, che analizza le dipendenze presenti nel classpath e configura automaticamente i Bean necessari;
- `@ComponentScan`: scansiona il *package* corrente alla ricerca di classi annotate con gli stereotipi (`@Component`, `@Service`, ...), registrandole automaticamente come Bean.

Grazie a questa singola annotazione, l'intera applicazione viene rilevata e collegata automaticamente all'avvio, senza configurazione esplicita.

5.3.2 Modello dei dati

L'entità `Customer`, mappata sulla tabella `customers` di PostgreSQL:

```
@Entity
@Table(name = "customers")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Customer {
    @Id
    private Long id;
    @Column(name = "full_name", nullable = false)
    private String fullName;
    @Column(name = "email")
    private String email;
    @Column(name = "city")
    private String city;
}
```

Codice 5.2: Customer

Le annotazioni Jakarta Persistence (JPA), elaborate da Hibernate a *runtime*, definiscono il *mapping* tra la classe Java e la tabella del *database*:

- `@Entity`: dichiara che la classe è un'entità persistente, cioè un oggetto il cui stato può essere salvato e recuperato dal database tramite JPA;
- `@Table(name = "customers")`: specifica esplicitamente il nome della tabella a cui l'entità è mappata (in assenza di questa annotazione, JPA userebbe per convenzione il nome della classe);
- `@Id`: indica il campo che costituisce la chiave primaria della tabella;
- `@Column(...)`: mappa un campo della classe su una colonna specifica, permettendo di personalizzarne il nome e i vincoli (ad esempio `nullable = false`, che impone che il campo non possa essere nullo).

Le annotazioni Lombok, agiscono in fase di compilazione: il compilatore, tramite un *annotation processor*, genera automaticamente il codice sorgente corrispondente, che viene poi incluso nel *bytecode* risultante, senza che sia visibile nel file:

- `@Getter` / `@Setter`: generano rispettivamente i metodi *getter* (`getFullName()`, `getEmail()`, ...) e i metodi *setter* (`setFullName(...)`, ...) per tutti i campi della classe;
- `@NoArgsConstructor`: genera un costruttore vuoto (senza argomenti), richiesto obbligatoriamente dalla specifica JPA, poiché Hibernate deve poter istanziare l'entità tramite *reflection* prima di popolarne i campi;
- `@AllArgsConstructor`: genera un costruttore che accetta un parametro per ciascun campo della classe, utile per la creazione di istanze (ad esempio nei test).

I DTO che rappresentano i contratti dei messaggi Kafka seguono un'impostazione analoga, con l'aggiunta di un'ulteriore annotazione Lombok:

```
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@ToString
public class OrderEvent {
    private String orderId;
    private Long customerId;
    private String product;
    private BigDecimal amount;
}
```

Codice 5.3: OrderEvent

```
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@ToString
public class EnrichedOrder {
    private String orderId;
    private Long customerId;
    private String customerFullName;
    private String customerEmail;
    private String customerCity;
    private String product;
    private BigDecimal amount;
    private Instant enrichedAt;
}
```

Codice 5.4: EnrichedOrder

- `@ToString`: genera un'implementazione del metodo `toString()` che elenca automaticamente il nome e il valore di ciascun campo, particolarmente utile in fase di *logging* (come si vedrà in `OrderConsumer`, dove l'oggetto `OrderEvent` viene loggato direttamente).

5.3.3 Accesso ai dati

L'accesso a PostgreSQL è realizzato tramite un *repository* Spring Data JPA, che estende `JpaRepository` senza necessità di implementazioni aggiuntive:

```
public interface CustomerRepository
    extends JpaRepository<Customer, Long> {

}
```

Codice 5.5: CustomerRepository

Qui non compaiono annotazioni esplicite: è sufficiente che l'interfaccia estenda `JpaRepository<Customer, Long>` (dove `Customer` è il tipo dell'entità e `Long` il tipo della sua chiave primaria) perché Spring Data JPA generi automaticamente, a *runtime*, un'implementazione proxy dell'interfaccia, fornendo di *default* i metodi CRUD più comuni (`findById`, `save`, `deleteById`, `findAll`, ...).

Questo meccanismo, che non richiede annotazioni perché il compor-

tamento è determinato dall'interfaccia ereditata, è ciò che permette al progetto di interrogare PostgreSQL senza scrivere alcuna query SQL esplicita.

5.3.4 Logica di enrichment

Il cuore applicativo del pattern Content Enricher è la classe `EnrichmentService`:

```
@Slf4j
@Service
@RequiredArgsConstructor
public class EnrichmentService {

    private final CustomerRepository customerRepository;

    @Transactional(readOnly = true)
    public EnrichedOrder enrich(OrderEvent orderEvent) {
        Customer customer = customerRepository
            .findById(orderEvent.getCustomerId())
            .orElseThrow(() ->
                new CustomerNotFoundException(orderEvent.getCustomerId()));

        return new EnrichedOrder(
            orderEvent.getOrderId(),
            orderEvent.getCustomerId(),
            customer.getFullName(),
            customer.getEmail(),
            customer.getCity(),
            orderEvent.getProduct(),
            orderEvent.getAmount(),
            Instant.now()
        );
    }
}
```

Codice 5.6: EnrichmentService

Questa classe è composta da quattro annotazioni:

- `@Slf4j`: inietta automaticamente un campo statico `log` di tipo `Logger`, consente di scrivere direttamente `log.debug(...)`.
- `@Service`: è una specializzazione di `@Component`, che identifica la classe come componente contenente logica di business. Dal punto di vista del container Spring il comportamento è identico a `@Component` (la classe viene rilevata dal `@ComponentScan` e registrata come Bean), ma l'uso di `@Service` comunica esplici-

tamente l'intento architetturale della classe, migliorando la leggibilità del codice.

- `@RequiredArgsConstructor`: genera un costruttore con un parametro per ciascun campo dichiarato `final` (in questo caso `customerRepository`). Combinata con l'assenza di un costruttore esplicito, questa annotazione realizza la *Dependency Injection* tramite costruttore: Spring, individuando un unico costruttore, lo invoca automaticamente iniettando il Bean `CustomerRepository` disponibile nel *container*. Questo approccio è preferibile rispetto all'iniezione tramite campo (`@Autowired`), poiché rende le dipendenze esplicite, immutabili e facilmente testabili.
- `@Transactional(readOnly = true)`: delimita una transazione database attorno all'esecuzione del metodo `enrich`. Il parametro `readOnly = true` segnala esplicitamente che il metodo esegue solo operazioni di lettura, permettendo a Hibernate di applicare alcune ottimizzazioni (ad esempio disabilitando il *dirty checking*, ovvero il meccanismo che traccia le modifiche alle entità per determinare se sia necessario un aggiornamento sul database). Sebbene nel caso specifico l'operazione compiuta sia una singola *query*, l'uso esplicito di `@Transactional` rende il comportamento robusto anche a future estensioni del metodo che coinvolgano più interrogazioni.

5.3.5 Consumer

Il *listener* Kafka orchestra l'intero flusso:

```
@Slf4j
@Component
@RequiredArgsConstructor
public class OrderConsumer {

    private final EnrichmentService enrichmentService;
    private final EnrichedProducer producer;

    @KafkaListener(
        topics = "${app.kafka.topic.input}",
        groupId = "${spring.kafka.consumer.group-id}"
    )
    public void consume(ConsumerRecord<String, OrderEvent> record,
        Acknowledgment ack) {
        String key = record.key();
        OrderEvent orderEvent = record.value();

        try {
            EnrichedOrder enriched = enrichmentService
                .enrich(orderEvent);
            producer.sendEnriched(key, enriched);
            ack.acknowledge();

        } catch (CustomerNotFoundException e) {
            producer.sendToDlq(key, orderEvent);
            ack.acknowledge();

        } catch (Exception e) {

        }
    }
}
```

Codice 5.7: OrderConsumer

- **@Component**: indica che la classe deve essere gestita come Bean dal *container*. Usato, invece di **@Service**, perché la classe non contiene logica di business.
- **@KafkaListener**: è l'annotazione che realizza concretamente il modello *listener* descritto in 4.6.3. Applicata a un metodo, ne fa un punto di ricezione automatica dei messaggi pubblicati sul topic specificato:
 - **topics = "\${app.kafka.topic.input}"**: il nome del topic da ascoltare non è scritto staticamente, ma iniettato da configura-

zione tramite la sintassi *placeholder* `$...`, che Spring risolve leggendo la proprietà corrispondente da `application.yml` (in questo caso `orders-input`). Questo disaccoppia il codice dal valore concreto del nome del topic, rendendolo configurabile per ambiente (sviluppo, test, produzione) senza modifiche al codice sorgente;

- `groupId = "$spring.kafka.consumer.group-id"`: allo stesso modo, il *consumer group* (introdotto in 4.5.3) è delegato in configurazione.

Il metodo annotato accetta come parametri un `ConsumerRecord<String, OrderEvent>`, l'involucro nativo del *client* Kafka che espone, oltre al *payload* deserializzato, anche metadati quali topic, partizione e *offset*, e un oggetto `Acknowledgment`, che rappresenta il meccanismo tramite cui il metodo conferma esplicitamente l'elaborazione del messaggio (`ack.acknowledge()`), coerentemente con la modalità `ack-mode: manual` configurata in `application.yml`.

La `CustomerNotFoundException` (errore atteso) viene gestita instradando il messaggio sulla DLQ e confermando comunque l'*offset*. Un'eccezione imprevista, invece, lascia l'*offset* non confermato, così da consentire un nuovo tentativo di elaborazione al riavvio del *consumer*.

5.3.6 Producer

La classe `EnrichedProducer` si occupa della pubblicazione, sia sul topic di *output* sia sulla DLQ:

```

@Slf4j
@Component
@RequiredArgsConstructor
public class EnrichedProducer {

    private final KafkaTemplate<String, Object> kafkaTemplate;

    @Value("${app.kafka.topic.output}")
    private String outputTopic;

    @Value("${app.kafka.topic.dlq}")
    private String dlqTopic;

    public void sendEnriched(String key, EnrichedOrder event) {
        kafkaTemplate.send(outputTopic, key, event)
            .whenComplete((result, ex) -> { });
    }

    public void sendToDlq(String key, Object originalPayload) {
        kafkaTemplate.send(dlqTopic, key, originalPayload)
            .whenComplete((result, ex) -> { });
    }
}

```

Codice 5.8: EnrichedProducer

Rispetto alle classi precedenti, qui compare una nuova annotazione:

- `@Value("${app.kafka.topic.output}")`: a differenza di `@KafkaListener`, dove il *placeholder* è risolto internamente all'annotazione stessa, `@Value` è un'annotazione generica che può essere applicata a un campo per iniettarvi direttamente un valore letto dalla configurazione. Il campo `outputTopic` viene così valorizzato automaticamente da Spring all'atto della creazione del Bean, senza bisogno di leggerlo esplicitamente da un oggetto `Environment`.

Il metodo `sendEnriched` utilizza `KafkaTemplate<String, Object>`, il componente ad alto livello fornito da Spring Kafka per la produzione di messaggi (analogo, sul lato *producer*, a ciò che `@KafkaListener` rappresenta sul lato *consumer*).

Il metodo `send(...)` restituisce un `CompletableFuture`, sul quale viene registrata tramite `whenComplete(...)` una *callback* asincrona per il *logging* dell'esito della pubblicazione, *pattern* coerente con la natura non bloccante dell'invio dei messaggi Kafka.

5.3.7 Sintesi delle annotazioni utilizzate

La tabella seguente riassume, per libreria di provenienza, tutte le annotazioni impiegate nel progetto e la loro funzione:

Annotazione	Libreria	Funzione
<code>@SpringBootApplication</code>	Spring Boot	<i>auto-configuration</i> + <i>component scan</i>
<code>@Component</code>	Spring Core	registra un Bean generico
<code>@Service</code>	Spring Core	registra un Bean di logica di business
<code>@RequiredArgsConstructor</code>	Lombok	costruttore per i campi <i>final</i> -> <i>Dependency Injection</i>
<code>@Transactional</code>	Spring	delimita una transazione database
<code>@KafkaListener</code>	Spring Kafka	registra un <i>listener</i> su un topic
<code>@Value</code>	Spring Core	inietta un valore da configurazione
<code>@Entity</code> / <code>@Table</code> / <code>@Id</code> / <code>@Column</code>	Jakarta Persistence	<i>mapping</i> oggetto-relazionale
<code>@Getter</code> / <code>@Setter</code>	Lombok	generano accessori
<code>@NoArgsConstructor</code> / <code>@AllArgsConstructor</code>	Lombok	generano costruttori
<code>@ToString</code>	Lombok	genera il metodo <i>toString()</i>
<code>@Slf4j</code>	Lombok	inietta un <i>logger</i> SLF4J

Tabella 5.2: Tabella delle annotazioni

Questa distinzione per provenienza è utile dal punto di vista didattico:

- le annotazioni Lombok agiscono a livello di compilazione, generando codice altrimenti scritto manualmente;
- le annotazioni Jakarta Persistence sono elaborate da Hibernate a *runtime* per il *mapping* oggetto-relazionale;

- le annotazioni Spring (Core e Kafka) sono invece elaborate dal container IoC all'avvio dell'applicazione, per la registrazione dei Bean, l'iniezione delle dipendenze e la configurazione dei *listener*.

5.4 Progettazione di una pipeline in Apache Flink

Sulla base dei concetti presentati in 4.7.4, la *pipeline* realizzata con Spring: lettura da `orders-input`, arricchimento tramite PostgreSQL, scrittura su `orders-output` o instradamento verso `orders-input-dlq`, si progetta, in Flink, riutilizzando gli stessi contratti dei messaggi definiti in 5.2.

5.4.1 Sorgente e formato dei dati

Il punto di ingresso della pipeline è un **Kafka Source**, il connettore nativo che Flink mette a disposizione per consumare un *topic* Kafka come *stream* di eventi, deserializzando ciascun *record* nel medesimo formato `OrderEvent` già impiegato nell'implementazione Spring (5.2.1).

5.4.2 Fase di enrichment

La logica di arricchimento, equivalente a quella incapsulata da `EnrichmentService` (5.3.4), si realizza tramite un operatore di tipo **Async I/O** (4.7.3), che interroga in modo non bloccante la tabella `customers` su PostgreSQL a partire dal `customerId` presente in ciascun `OrderEvent`, e produce in *output* il medesimo `EnrichedOrder` già definito nel progetto.

5.4.3 Gestione degli errori

La gestione dei clienti non trovati, realizzata in Spring tramite `CustomerNotFoundException` e il pattern Dead Letter Channel (4.4.2), si realizza in Flink tramite un **Side Output**: un canale di uscita secondario, associato all'operatore di *enrichment*, verso cui si instradano gli eventi per cui il *lookup* su PostgreSQL non produce alcun risultato, stesso principio architetturale, meccanismo nativo della piattaforma anziché *try/catch* applicativo.

5.4.4 Destinazione e garanzie di consegna

Il messaggio arricchito e l'eventuale messaggio instradato sul canale di errore vengono pubblicati rispettivamente su `orders-output` e `orders-input-dlq` tramite un **Kafka Sink**. Sul piano della semantica di consegna (4.5.5), il *checkpointing* periodico dell'intero stato della *pipeline*, incluso l'*offset* Kafka in lettura, garantisce una semantica *exactly-once end-to-end* se combinato con un Kafka Sink transazionale, una garanzia più forte rispetto alla semantica *at-least-once* realizzata nell'implementazione Spring.

5.5 Strumenti per i test di performance

Dato lo *stack* tecnologico descritto in 5.1, la metodologia di valutazione definita in 4.8 si realizza con i seguenti strumenti, in gran parte già presenti nell'ambiente di sviluppo predisposto per il progetto.

5.5.1 Generazione del carico

Il carico si genera con `kafka-producer-perf-test.sh`, lo script incluso nella distribuzione Kafka già containerizzata in Docker, che pubblica un volume controllato di eventi `OrderEvent` su `orders-input`, con parametri configurabili quali: numero totale di messaggi, dimensione del *payload* e *Throughput* massimo. Per lo scenario con percentuale variabile di messaggi non arricchibili (si veda 4.8.2), il carico si genera invece con uno *script* dedicato, che pubblica `customerId` validi e non validi secondo una distribuzione controllata.

5.5.2 Osservazione del comportamento del sistema

Il *consumer lag* si osserva in due modi complementari:

- tramite il comando nativo `kafka-consumer-groups.sh --describe`;
- tramite **Kafka UI**, già incluso nell'ambiente Docker Compose del progetto, che espone questa informazione in tempo reale.

5.5.3 Osservabilità applicativa

Il progetto include tra le dipendenze Maven `spring-boot-starter-actuator`, che espone *endpoint* HTTP standard per il monitoraggio dello stato

dell'applicazione. Tale dipendenza si integra nativamente con **Micrometer**¹, libreria di strumentazione collegabile a **Prometheus**² per la raccolta storica delle metriche e a **Grafana**³ per la loro visualizzazione tramite *dashboard*, permettendo di correlare temporalmente il carico generato con le metriche applicative e di sistema.

5.5.4 Ambiente di test

L'ambiente Docker Compose predisposto per il progetto prevede un singolo nodo Kafka in modalità KRaft e una singola istanza PostgreSQL. Qualunque risultato ottenuto in questo ambiente si legge come indicativo della bontà architetturale del *middleware*, più che come misura assoluta delle prestazioni ottenibili in un *deployment* distribuito su più nodi.

¹Micrometer - URL: <https://micrometer.io/>

²Prometheus - URL: <https://prometheus.io/>

³Grafana - URL: <https://grafana.com/>

Capitolo 6

Conclusioni

6.1 Sintesi del lavoro svolto

Il presente lavoro ha affrontato lo studio, la progettazione e l'implementazione di un *middleware* basato sul pattern Content Enricher, sviluppato secondo l'obiettivo di tirocinio descritto nel Capitolo 3.

Il sistema realizzato legge eventi da un *topic* Apache Kafka sorgente, li arricchisce con dati letti da PostgreSQL e pubblica il risultato su un *topic* di destinazione, gestendo esplicitamente i casi di dato mancante tramite il pattern Dead Letter Channel. L'intera implementazione è realizzata tramite l'ecosistema Spring (Spring Boot, Spring Data JPA, Spring Kafka), come descritto nel Capitolo 5. Il Capitolo 4 ha inoltre inquadrato queste scelte implementative all'interno della teoria dei *Middleware* e degli *Enterprise Integration Patterns*, e ha esteso l'analisi, a livello concettuale, al confronto con Apache Flink come paradigma alternativo di *stream processing*.

Gli obiettivi obbligatori O01-O03 definiti in fase di analisi dei requisiti risultano soddisfatti, come riportato nelle relative tabelle di tracciamento; gli obiettivi desiderabili e facoltativi relativi al confronto con Flink e ai test di performance sono discussi nella sezione 6.3.

6.2 Conoscenze acquisite

Il percorso di tirocinio ha permesso di approfondire diverse competenze tecniche, in parte specifiche dello *stack* tecnologico impie-

gato, in parte più generali rispetto alla progettazione di sistemi *middleware*:

- l'ecosistema **Spring**, in particolare il modello di *Dependency Injection* e *Inversion of Control* alla base di Spring Boot, l'astrazione *repository* offerta da Spring Data JPA e il modello *listener* di Spring Kafka, discussi nel dettaglio nel Capitolo 5;
- i concetti fondamentali di **Apache Kafka**: *topic*, partizione, *offset*, *consumer group*, semantica di consegna e il loro impatto sulle scelte architetturali di un sistema *event-driven*, approfonditi nel Capitolo 4;
- gli **Enterprise Integration Patterns**, in particolare Content Enricher e Dead Letter Channel, come vocabolario condiviso per descrivere problemi ricorrenti nell'integrazione di sistemi distribuiti, applicati concretamente nell'implementazione del *middleware*;
- l'uso di **Docker** e **Docker Compose** per la definizione di un ambiente di sviluppo riproducibile, comprendente Apache Kafka, PostgreSQL e i relativi strumenti di ispezione;
- i concetti di base della piattaforma **Apache Flink**: Kafka Source, Kafka Sink, Side Output, utili a comprendere le differenze tra un modello di elaborazione *event-driven* a *listener* e uno *stream processing* nativo.

6.3 Limiti e sviluppi futuri

Due obiettivi definiti in fase di analisi dei requisiti non sono stati portati a termine, per la priorità assegnata allo sviluppo dell'implementazione principale nel tempo a disposizione del tirocinio: l'implementazione equivalente in **Apache Flink** (D02, RFD-07, 3.5) e l'esecuzione dei **test di performance** (F01, RFF-08, 3.6).

Per entrambi, il presente lavoro non si è fermato alla sola indicazione dell'obiettivo mancato, ma ha definito una base pronta all'esecuzione:

- riguardo ad **Apache Flink**, il Capitolo 4 presenta i concetti teorici della piattaforma e un confronto concettuale con l'approccio Spring adottato (4.7.4), mentre il Capitolo 5 ne definisce la

progettazione concreta di una pipeline equivalente, sugli stessi contratti dei messaggi e sugli stessi *topic* già implementati (5.4);

- riguardo ai **test di performance**, il Capitolo 4 definisce le metriche di valutazione e la metodologia di test (4.8), mentre il Capitolo 5 indica gli strumenti concreti con cui tali test si eseguirebbero, dato lo *stack* tecnologico impiegato (5.5).

Una naturale prosecuzione del lavoro consiste quindi nel realizzare l'implementazione Flink secondo la progettazione già definita, ed eseguire su entrambe le implementazioni, quella basata su Spring e quella basata su Flink, la medesima metodologia di test di performance, permettendo così un confronto fondato su dati concreti.

Acronimi

ACID Atomicity Consistency Isolation Durability. 34,

API Application Programming Interface. 24, I,

CRUD Create Read Update Delete. 41,

DBMS Database Management System. 33, 34,

DLQ Dead Letter Queue. 15, 25, 37, 45,

DSL Domain-Specific Language.

DTO Data Transfer Object. 16, 40,

EIP Enterprise Integration Patterns. 14, 16,

HTTP Hypertext Transfer Protocol. 49,

IDE Integrated Development Environment. II,

IoC Inversion of Control. 21, 48, III,

IoT Internet of Things. 1,

JAR Java Archive. 29,

JDBC Java Database Connectivity. 29,

JPA Java Persistence API. 25, 29, 39, 40,

JVM Java Virtual Machine. 28, 29,

MOM Message-Oriented Middleware. 13,

NoSQL Not only SQL. 34,

POM Project Object Model. 29,

RPC Remote Procedure Call. 13,

SLF4J Simple Logging Facade for Java. 47,

SQL Structured Query Language. 34, 42, I,

XML eXtensible Markup Language. 29, 30,

Glossario

Annotazioni nel linguaggio di programmazione Java, un'annotazione è una forma di metadati sintattici che può essere aggiunta al codice sorgente Java, proprio come un attributo. È possibile annotare classi, metodi, variabili, parametri e pacchetti Java. 38

Bean classi che incapsulano uno o più oggetti in un unico oggetto standardizzato (il *bean*). Questa standardizzazione consente di gestire i *bean* in modo più generico, facilitando il riutilizzo del codice e l'introspezione. Ciò permette, a sua volta, di trattare i *bean* come componenti software e di manipolarli visivamente tramite editor e IDE senza bisogno di alcuna configurazione iniziale né di conoscere alcun dettaglio di implementazione interna.. 38, 42-44, 46-48

Checkpointing tecnica che garantisce la tolleranza ai guasti nei sistemi informatici. Consiste nel salvare un'istantanea dello stato di un'applicazione, in modo che questa possa riavviarsi da quel punto in caso di guasto. Ciò è particolarmente importante per le applicazioni a esecuzione prolungata che vengono eseguite in sistemi informatici soggetti a guasti. 24, III

Dependency Injection tecnica di programmazione in cui un oggetto o una funzione riceve altri oggetti o funzioni di cui ha bisogno, anziché crearli internamente. La *Dependency Injection* mira a separare gli aspetti relativi alla creazione degli oggetti da

quelli relativi al loro utilizzo, portando a programmi con un accoppiamento debole. Questo modello garantisce che un oggetto o una funzione che desideri utilizzare un determinato servizio non debba sapere come creare tali servizi. Al contrario, al "cliente" ricevente (oggetto o funzione) vengono fornite le sue dipendenze da un codice esterno (un "iniettore"), di cui esso non è a conoscenza. 23, 43, 47, 52, III

Hibernate strumento di mappatura oggetto-relazionale per il linguaggio di programmazione Java. Fornisce un framework per la mappatura di un modello di dominio orientato agli oggetti su un database relazionale. Hibernate risolve i problemi di disallineamento oggetto-relazionale sostituendo gli accessi diretti e persistenti al database con funzioni di gestione degli oggetti di alto livello. 34, 39, 40, 43, 47

Inversion of Control espressione adottata separatamente dalla comunità dei programmatori Java per riferirsi specificatamente ai modelli di *Dependency Injection* (il passaggio di servizi agli oggetti che ne hanno bisogno) che si verificano con i "container IoC" nei *framework* Java come Spring. In questo senso, si riferisce al fatto di concedere al *framework* il controllo sulle implementazioni delle dipendenze utilizzate dagli oggetti dell'applicazione, piuttosto che al significato originario di concedere al *framework* il flusso di controllo. 21, 52, I

Kafka Sink connettore nativo offerto da Apache Flink per la scrittura di un flusso di eventi (*stream*) su un Topic Kafka di destinazione. Rappresenta il punto di uscita di una *pipeline* Flink verso Kafka, complementare al Kafka Source, e può essere configurato con garanzie di consegna diverse (*at-least-once* o *exactly-once*), quest'ultima ottenibile in combinazione con il meccanismo di *Checkpointing* della piattaforma. 49, 52

Kafka Source connettore nativo offerto da Apache Flink per la lettura di un Topic Kafka come flusso continuo di eventi (*stream*). Si occupa della deserializzazione dei singoli messaggi nel formato atteso dall'applicazione e della gestione dell'*offset* di lettura, rappresentando il punto di ingresso di una *pipeline* Flink alimentata da Kafka. 48, 52, III

Lombok libreria Java che si integra automaticamente con l'editor e gli strumenti di compilazione, arricchendo il codice Java.. 30, 39, 40, 47

Middleware software che funge da strato intermedio tra le applicazioni e le componenti sottostanti, come ad esempio sistemi operativi, *database* o hardware, facilitando la comunicazione, l'integrazione e la gestione delle risorse nei sistemi distribuiti. Il suo ruolo primario è quello di nascondere la complessità dell'infrastruttura sottostante, garantendo interoperabilità, scalabilità e sicurezza. iv, 3

Open Source software distribuito, generalmente in via gratuita, sotto i termini di una licenza open source. 24, 31, 34

Side Output meccanismo offerto da Apache Flink che consente a un operatore di una *pipeline* di emettere, oltre al flusso di uscita principale, uno o più flussi di uscita secondari (*side output*), tipicamente utilizzati per instradare eventi che non possono essere elaborati correttamente verso un canale separato, senza interrompere l'elaborazione del flusso principale. 48, 52

Throughput La velocità di trasmissione (Throughput) indica la velocità di consegna dei messaggi su un canale di comunicazione all'interno di una rete di comunicazione. I dati contenuti in questi messaggi possono essere trasmessi tramite collegamenti fisici o logici, oppure attraverso nodi di rete. La velocità di trasmissione rappresenta il consumo di larghezza di banda digitale. La velocità effettiva di un sistema di comunicazione può essere influenzata da vari fattori, tra cui i limiti del mezzo fisico sottostante, la potenza di elaborazione disponibile dei componenti del sistema, il comportamento dell'utente finale. 17, 25, 35, 49, v

Topic Kafka categorie utilizzate per organizzare i messaggi. Ogni topic ha un nome unico per l'intero cluster Kafka. I messaggi vengono inviati e letti da argomenti specifici. I produttori scrivono dati negli argomenti e i consumatori leggono dati dagli argomenti. In Kafka, gli argomenti sono partizionati e replicati attraverso i broker per tutta l'implementazione. I broker si

referiscono a ciascuno dei nodi di un cluster Kafka. Le partizioni sono importanti perché permettono la parallelizzazione degli argomenti, consentendo un elevato *Throughput* dei messaggi. iv, 3, 9, 10, 36, III

Visual Studio Code editor di codice sorgente sviluppato da Microsoft per Windows, Linux e macOS. È un software libero e gratuito per uso personale e commerciale, anche se la versione ufficiale è sotto una licenza proprietaria. Permette di installare estensioni che aggiungono ulteriori funzionalità.

Bibliografia

Siti web

Annotation (Jakarta Persistence) - URL: <https://jakarta.ee/learn/docs/jakartaee-tutorial/current/persist/persistence-intro/persistence-intro.html> - 5.3

Annotation (Java) - URL: https://en.wikipedia.org/wiki/Java_annotation - 5.3

Annotation (Lombok) - URL: <https://projectlombok.org/features/> - 5.3

Annotation (Spring) - URL: <https://docs.spring.io/spring-framework/reference/testing/annotations.html> - 5.3

Apache Flink - URL: <https://flink.apache.org/> - 4.7, 5.4

Apache Maven - URL: <https://maven.apache.org/> - 5.1.1.2

Apache Kafka - URL: <https://kafka.apache.org/> - 4.5, 5.1.3.3

Content Enricher - URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/DataEnricher.html> - 4.4.1

Dead Letter Channel - URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/DeadLetterChannel.html> - 4.4.2

Docker - URL: <https://www.docker.com/> - 5.1.3.1

Enterprise Integration Patterns - URL: <https://www.enterpriseintegrationpatterns.com/> - 4.4

Event Streaming - URL: <https://www.ibm.com/think/topics/event-streaming> - 4.5.2

Java - URL: <https://www.java.com/> - 5.1.1.1

Kafka Sink - URL: <https://kafka.apache.org/36/javadoc/org/apache/kafka/connect/sink/SinkConnector.html> - 5.4.4

Kafka Source - URL: <https://kafka.apache.org/36/javadoc/org/apache/kafka/connect/source/SourceConnector.html> - 5.4.1

Message Channel - URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/MessageChannel.html> - 4.4.3

Message Translator - URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/MessageTranslator.html> - 4.4.3

Middleware - URL: <https://en.wikipedia.org/wiki/Middleware> - 4

Middleware - URL: [https://en.wikipedia.org/wiki/Middleware_\(distributed_applications\)](https://en.wikipedia.org/wiki/Middleware_(distributed_applications)) - 4

Point-to-Point Channel - URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/PointToPointChannel.html> - 4.4.3

PostgreSQL - URL: <https://www.postgresql.org/> - 5.1.3.2

Side Output - URL: https://nightlies.apache.org/flink/flink-docs-stable/docs/dev/datastream/side_output/ - 5.4.3

Spring - URL: <https://spring.io/> - 5.1.2.1

Spring Boot - URL: <https://spring.io/projects/spring-boot> - 4.6.1

Spring Boot - URL: https://en.wikipedia.org/wiki/Spring_Boot - 4.6.1

Spring Data JPA - URL: <https://spring.io/projects/spring-data-jpa> - 4.6.2

Spring Framework - URL: https://en.wikipedia.org/wiki/Spring_Framework

- 4.6

Spring Kafka - URL: <https://spring.io/projects/spring-kafka> - 4.6.3